

Assam Don Bosco University | MCA 1 – 2026 AUTUMN

LAFSys (Campus Lost and Found System)

Complete Project Details & System Knowledge Document

1. Project Overview

LAFSys is a Java-based Desktop Graphical User Interface (GUI) application designed to modernize and streamline the management of lost and found items on a university campus. Instead of relying on manual logbooks or physical notice boards, LAFSys provides a digital platform where students and staff can quickly report lost items, log found items, and search through a centralized database.

2. Problem Statement & Proposed Solution

The Problem

In large university campuses, students frequently lose personal belongings (IDs, electronics, books, keys). The traditional approach relies on handing items to security desks and writing details in a physical register. This method is highly inefficient: it is prone to human error, difficult to search through, and lacks real-time updates. Students often waste days checking multiple departments for their lost belongings.

The Solution

LAFSys solves this by digitizing the process into a single, user-friendly desktop application. It ensures that data is standardized, easily searchable via keywords or categories, and visually accessible through an intuitive tabbed interface. Status tracking allows an item to transition from "Active" to "Claimed", preventing duplicate claims and keeping records accurate.

3. Core Features & Functionalities

- Post a Lost Item:** Users can input details of an item they have lost, including Title, Description, Category, Date, Location, and their Name.
- Post a Found Item:** Users who find misplaced items can log them into the system so the rightful owner can search for them.
- Categorized Display:** Users can view separated lists of "All Lost Items" and "All Found Items" formatted cleanly on the screen.
- Keyword Search:** A search bar allows users to instantly filter the database for specific words (e.g., searching "iPhone" will pull up all related records).
- Item Claiming System:** Users can claim an item using its unique ID. The system validates the ID and changes the item's status to "Claimed", securing the record.

4. Technical Stack & Specifications

Technology	Usage in LAFSys
Java (JDK 8+)	The core programming language used for all logic and object-oriented structure.
Java Swing	The native Java library used to build the Graphical User Interface (Windows, Buttons, Text Fields).

Java Collections (ArrayList)	Used as the primary runtime database to store, retrieve, and manipulate lists of item records dynamically.
Event-Driven Programming	Utilizing <code>ActionListeners</code> to execute code only when the user interacts with the GUI (e.g., clicking a button).

5. System Architecture (The 3 Pillars)

The application is designed using a modular, Object-Oriented approach. The code is divided into three distinct Java files, each serving a specific purpose. This separation makes the code clean, readable, and easy to maintain.

Pillar 1: The Data Model (Item.java)

This class acts as a blueprint for a single real-world object (a lost or found item). It uses **Encapsulation** by keeping its variables private and exposing them through public getter and setter methods.

- **Attributes:** `itemId`, `title`, `description`, `category`, `datePosted`, `status`, `posterName`, `location`
- **Methods:** Constructor, Getters, Setters, and a custom `toString()` method to format the item nicely for the screen.

Pillar 2: The Business Logic (LostAndFoundBoard.java)

This class acts as the "Backend System". It manages the data storage and performs all calculations and searches. It does *not* care about the GUI.

- **Storage:** `ArrayList<Item> items` (Holds all items in memory).
- **Methods:** `addItem(Item item)`, `getAllLostItems()`, `searchKeyword(String key)`, `claimItem(int id)`.

Pillar 3: The User Interface (MainGUI.java)

This class contains the `main` method and builds the visual application. It takes user input from text boxes, passes it to the Business Logic (Pillar 2), and displays the results back to the user.

- **Components:** `JFrame` (Window), `JTabbedPane` (Tabs), `JTextField` (Input Boxes), `JButton` (Buttons).

6. GUI Layout & Event Flow

The GUI is structured using a `JTabbedPane`, allowing the user to switch between three main views without opening new windows:

Tab 1: Post Item

- **Layout:** A vertical form with labels and text fields for data entry. A dropdown box (`JComboBox`) lets the user select if the item is "Lost" or "Found".
- **Event Flow:** User fills fields → Clicks "Submit" → `ActionListener` reads the fields → Creates an `Item` object → Sends to Backend → Shows a Success Popup (`JOptionPane`).

Tab 2: View & Search Items

- **Layout:** A search bar at the top with a "Search" button. Two filter buttons below it ("View All Lost", "View All Found"). A large, non-editable text area (`JTextArea`) at the bottom to display results.
- **Event Flow:** User clicks "View All Lost" → Logic fetches lost items from `ArrayList` → Formats them into a giant `String` → Sets the text of the `JTextArea` .

Tab 3: Claim Item

- **Layout:** A simple, clean interface requiring only one input: the Item ID.
- **Event Flow:** User enters ID → Clicks "Claim" → Backend searches `ArrayList` for the ID → If found, changes status to "Claimed" and returns `true` → GUI shows a Success Popup (or Error popup if ID is fake).

7. Educational Value & OOP Concepts Demonstrated

This project serves as an excellent demonstration of core academic Java concepts required for MCA students:

- **Classes & Objects:** Instantiating multiple `Item` objects from a single class blueprint.
- **Encapsulation:** Protecting data inside the `Item` class and accessing it strictly via methods.
- **Data Structures:** Utilizing `ArrayList` to understand dynamic memory allocation, replacing standard fixed-size arrays.
- **Control Flow & Validation:** Using `if/else` statements and loops (`for-each`) to iterate through data and validate user inputs before processing.
- **GUI Event Handling:** Understanding how graphical operating systems wait for user actions (listeners) rather than executing line-by-line like a console app.